



Sisteme de numerație | Operații pe biți

Andrei Mihai Ciobanu

Algolymp Summer School

Baze de numerație

Reprezentarea numerelor în calculator

Conversii între baze

Operații pe biți

Aplicații

Funcții utile

Recapitulare

Resurse

Probleme de antrenament

Baze de numerație

Baze de numerație

Un număr se poate scrie în orice **bază** b folosind cifre de la 0 la $b - 1$:

Baza	Cifre	Exemplu	Sumă de puteri
10 (zecimal)	0 ... 9	425_{10}	$4 \cdot 10^2 + 2 \cdot 10 + 5$
2 (binar)	0, 1	$1101_2 = 13$	$2^3 + 2^2 + 2^0$
16 (hexa)	0 ... 9, A ... F	$FF_{16} = 255$	$15 \cdot 16 + 15$

- În hexazecimal:

$A = 10$, $B = 11$, $C = 12$, $D = 13$, $E = 14$, $F = 15$

- În C++: `0b1101` = binar, `0xFF` = hexazecimal

Reprezentarea numerelor în calculator

Cum stochează calculatorul numerele?

Calculatoarele lucrează doar cu **biți** — cifre care pot fi doar **0** sau **1**.

Orice număr întreg se reprezintă ca o **sumă de puteri ale lui 2**:

$$13 = 2^3 + 2^2 + 2^0 = 8 + 4 + 1$$

Reprezentare pe 8 biți (exemplu: 13)

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
0	0	0	0	1	1	0	1

- `int` (32 biți) → valori de la -2^{31} la $2^{31} - 1 \approx 2 \cdot 10^9$
- `long long` (64 biți) → valori de la -2^{63} la $2^{63} - 1 \approx 9 \cdot 10^{18}$

Conversii între baze

Conversie din baza 10 în baza b

Algoritmul: împărțim repetat la b și citim resturile de jos în sus.

Exemplu: 13_{10} în baza 2:

- $13 \div 2 = 6$ rest 1
- $6 \div 2 = 3$ rest 0
- $3 \div 2 = 1$ rest 1
- $1 \div 2 = 0$ rest 1
- $\Rightarrow 13_{10} = 1101_2$

```
1 // Returneaza cifrele in baza b, de la cel mai semnificativ
2 vector<int> toBase(int n, int b) {
3     vector<int> digits;
4     do {
5         digits.push_back(n % b);
6         n /= b;
7     } while (n > 0);
8     reverse(digits.begin(), digits.end());
9     return digits;
10 }
```

▷ *vector, reverse* — funcții din STL (vezi slide-ul Resurse)

Conversie din baza b în baza 10

Procesăm cifrele de la stânga la dreapta:

$$1101_2 = ((1 \cdot 2 + 1) \cdot 2 + 0) \cdot 2 + 1 = 13$$

Pas cu pas:

- $result = 1$
- $result = 1 \cdot 2 + 1 = 3$
- $result = 3 \cdot 2 + 0 = 6$
- $result = 6 \cdot 2 + 1 = 13 \checkmark$

```
1 // digits: cifrele in baza b, de la cel mai semnificativ
2 int fromBase(vector<int> digits, int b) {
3     int result = 0;
4     for (int i = 0; i < digits.size(); i++) {
5         result = result * b + digits[i];
6     }
7     return result;
8 }
```

Operații pe biți

Noțiuni de bază

Numerotăm biții unui număr **de la dreapta la stânga**, începând cu 0.

Exemplu: $x = 13 = 1101_2$

Indice bit	3	2	1	0
Bit	1	1	0	1
Valoare	8	4	2	1

Biții setați: 3, 2, 0 $\Rightarrow 2^3 + 2^2 + 2^0 = 8 + 4 + 1 = 13$

- **LSB** — bitul 0, cel mai din dreapta
- **MSB** — bitul cu cel mai mare indice setat

NOT — ~

Inversează **fiecare bit**: 0 devine 1, 1 devine 0.

p	$\sim p$
0	1
1	0

Exemplu: $a = 10 = 1010_2$

$a:$	1	0	1	0
$\sim a:$	0	1	0	1

```
1  int a    = 0b1010; // 10
2  int r    = ~a;      // -11
```

Pe 32 biți: $\sim a = -(a + 1)$, deci $\sim 10 = -11$

▷ *Complement față de 2 — modul în care calculatoarele stochează numerele negative (vezi slide-ul Resurse)*

AND — &

Rezultat 1 dacă **ambii** biți corespunzători sunt 1.

p	q	$p \& q$
0	0	0
0	1	0
1	0	0
1	1	1

Exemplu: $a = 1010_2$, $b = 1100_2$

$a:$	1	0	1	0
$b:$	1	1	0	0
$a \& b:$	1	0	0	0

```
1  int a = 0b1010, b = 0b1100;  
2  int r = a & b; // 0b1000 = 8
```

Rezultat **1** dacă **cel puțin unul** dintre biți este 1.

p	q	$p \mid q$
0	0	0
0	1	1
1	0	1
1	1	1

Exemplu: $a = 1010_2$, $b = 1100_2$

$a:$	1	0	1	0
$b:$	1	1	0	0
$a \mid b:$	1	1	1	0

```
1  int a = 0b1010, b = 0b1100;
2  int r = a | b; // 0b1110 = 14
```

XOR — ^

Rezultat **1** dacă biții sunt **diferiți**.

p	q	$p \oplus q$
0	0	0
0	1	1
1	0	1
1	1	0

Exemplu: $a = 1010_2$, $b = 1100_2$

$$\begin{array}{rcccc} a: & 1 & 0 & 1 & 0 \\ b: & 1 & 1 & 0 & 0 \\ \hline a \oplus b: & 0 & 1 & 1 & 0 \end{array}$$

```
1  int a = 0b1010, b = 0b1100;
2  int r = a ^ b; // 0b0110 = 6
```

$a \oplus a = 0$; $a \oplus 0 = a$; XOR e comutativ și asociativ

Deplasări pe biți — Left Shift și Right Shift

```
1 int x = 1;           // 0b000000001
2
3 int a = x << 3;      // 0b00001000 = 8    (x * 2^3)
4 int b = x >> 0;      // 0b000000001 = 1
5
6 int y = 12;          // 0b00001100
7 int c = y << 1;      // 0b00011000 = 24    (y * 2)
8 int d = y >> 1;      // 0b00000110 = 6     (y / 2)
9 int e = y >> 3;      // 0b00000001 = 1     (y / 8)
```

- $x \ll k = x \cdot 2^k$ (deplasare la stânga)
- $x \gg k = \lfloor x/2^k \rfloor$ (deplasare la dreapta)
- **Atenție:** $1 \ll k$ dă overflow pentru $k \geq 31$ pe `int`
→ folosim `1LL << k` sau `1LL >> k` când lucrăm cu `long long`

Aplicații

Operații uzuale pe biți

```
1  // Verificam daca al k-lea bit este setat
2  bool isSet(int x, int k) {
3      return (x >> k) & 1;
4  }
5
6  // Caz special: x impar <=> bitul 0 setat
7  bool isOdd(int x) {
8      return x & 1;
9  }
10
11 // Setam al k-lea bit (il facem 1)
12 int setBit(int x, int k) {
13     return x | (1 << k);
14 }
15
16 // Resetam al k-lea bit (il facem 0)
17 int clearBit(int x, int k) {
18     return x & ~(1 << k);
19 }
```

Aplicații — Continuare

```
1 // Flippuim al k-lea bit (0->1, 1->0)
2 int flipBit(int x, int k) {
3     return x ^ (1 << k);
4 }
5
6 // Ultimii k biti ai lui x (ex: k=3 => masca 0b0111)
7 int lastKBits(int x, int k) {
8     return x & ((1 << k) - 1);
9 }
10
11 // n e putere de 2 daca are exact un singur bit de 1
12 // n & (n-1) elimina cel mai din dreapta bit de 1
13 bool isPow2(int n) {
14     return n > 0 && (n & (n - 1)) == 0;
15 }
```

- Exemplu: $4 = 100_2$, $3 = 011_2$, $4 \& 3 = 0$ ✓
- **Atenție:** folosește `1LL << k` în loc de `1 << k` pentru `long long`

Funcții utile

Popcount — Numărul de biți de 1

```
1 // Implementare manuala: O(nr. biti)
2 int popcount(int x) {
3     int cnt = 0;
4     while (x > 0) { cnt += x & 1; x >>= 1; }
5     return cnt;
6 }
7
8 // Built-in GCC (O(1), foarte rapid)
9 int cnt = __builtin_popcount(x);    // pentru int
10 int cnt = __builtin_popcountll(x);  // pentru long long
```

- Exemplu: $\text{popcount}(13) = \text{popcount}(0b1101) = 3$
- Built-in-urile `__builtin_*` sunt specifice GCC — merg pe majoritatea compilatoarelor de concurs (g++), dar nu garantat pe MSVC

Leading zeros și Trailing zeros

- **Leading zeros:** biți de 0 de la stânga până la primul bit de 1
- **Trailing zeros:** biți de 0 de la dreapta până la primul bit de 1

```
1 // x = 0b00001100 = 12
2 // Leading zeros = 28 (int are 32 biti, primii 28 sunt 0)
3 // Trailing zeros = 2 (ultimii 2 biti sunt 0)
4
5 int lz = __builtin_clz(x); // leading zeros (int)
6 int tz = __builtin_ctz(x); // trailing zeros (int)
7 // Variante pentru long long: __builtin_clzll,
8 //                               __builtin_ctzll
9
10 // Atentie: __builtin_clz(0) si __builtin_ctz(0) sunt
11 //           nedefinite!
```

- Trailing zeros al lui x = indicele celui mai mic bit setat al lui x
- Exemplu: $\text{__builtin_ctz}(12) = \text{__builtin_ctz}(0b1100) = 2$

Lowest Set Bit — $x \& (-x)$

Lowest set bit = valoarea celui mai din dreapta bit de 1.

```
1 int lowestBit(int x) { return x & (-x); }  
2 // x = 12 = 0b1100 => lowestBit(12) = 0b0100 = 4
```

De ce funcționează? — Reprezentarea complement față de 2:

- $-x = \sim x + 1$ (inversăm toți biții, adăugăm 1)
- La adunarea lui 1, carry-ul se propagă exact până la primul bit de 1 din x
- Astfel $-x$ coincide cu x începând de la primul bit de 1 (inclusiv), restul e inversat
- $x \& (-x)$ păstrează doar acel bit comun

Exemplu: $x = 12 = 1100_2$, $-x = 0100_2$ (în 4 biți),
 $x \& (-x) = 0100_2 = 4$

Recapitulare

Recapitulare — Operații uzuale

Operație	Cod C++
Bitul k e setat?	<code>(x >> k) & 1</code>
x e impar?	<code>x & 1</code>
Setează bitul k	<code>x (1 << k)</code>
Resetează bitul k	<code>x & ~(1 << k)</code>
Flip bitul k	<code>x ^ (1 << k)</code>
Ultimii k biți	<code>x & ((1 << k) - 1)</code>
n e putere de 2?	<code>n > 0 && !(n & (n-1))</code>
Lowest set bit	<code>x & (-x)</code>
Număr de biți de 1	<code>__builtin_popcount(x)</code>
Trailing zeros	<code>__builtin_ctz(x)</code>

Resurse

Baze de numerație

- pbinfo.ro — Baze de numerație (RO)

Operații pe biți

- pbinfo.ro — Operații pe biți (RO, ușor de înțeles)
- cp-algorithms.com — Bit manipulation (EN)
- cppi.sync.ro — Material mai stufos de Marius Nicoli (RO)

Complement față de 2

- wikipedia.org — Two's complement (EN)

Probleme de antrenament

Ușoare

- **getbit** (pbinfo)
- **setbit0** (pbinfo)
- **setbit1** (pbinfo)
- **puterebiti** (pbinfo)

Medii

- **Base Bit Report** (AlgoCode)
- **addone** (pbinfo)
- **invbits** (pbinfo)